

Data Management 2 - Final Project

Student: Madan Kumar Kumar

Matriculation No.: 100004225

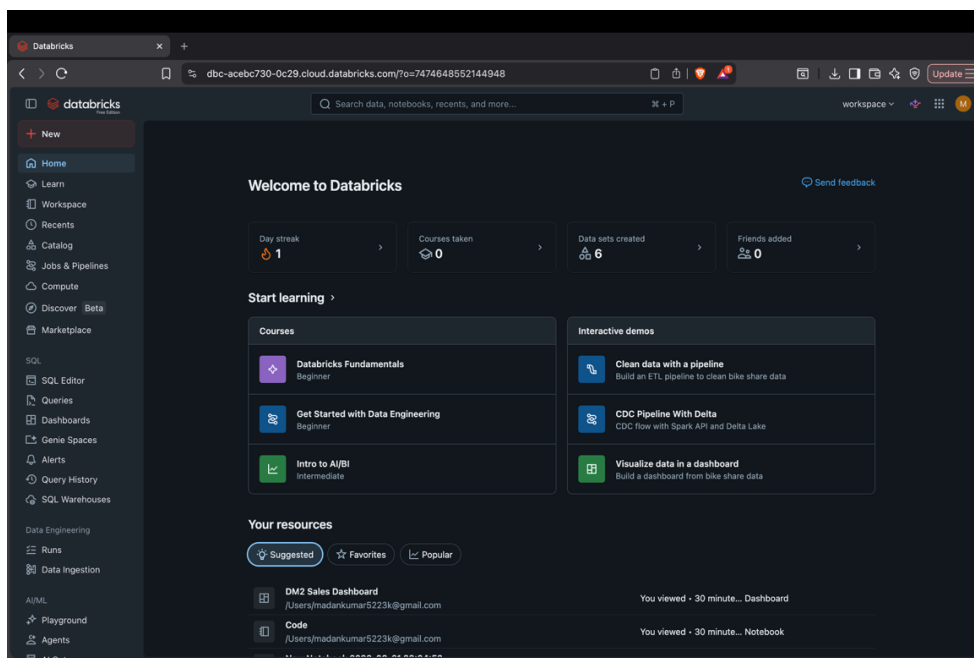
Course: Data Management 2 - Data Curation and Data Management

Instructor: Esam Sharaf - SRH Campus Hamburg

Data source: Chen, D. (2015). *Online Retail* [Dataset]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5BW33> (CC BY 4.0).

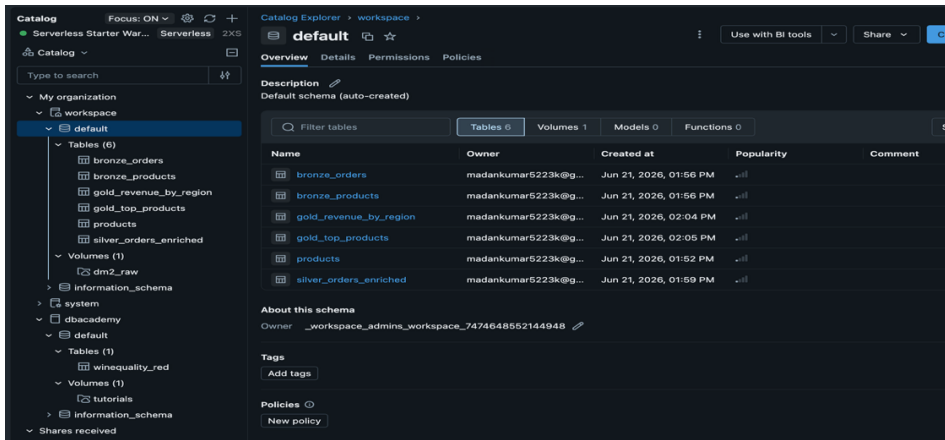
This document evidences the configuration, key pipeline stages, and outcomes of the project. The pipeline combines two different data source types (a Delta table and a streaming JSON folder) through a Bronze -> Silver -> Gold architecture on Databricks and reports two business objectives in an AI/BI dashboard. Full code is in the accompanying notebook; the data source is cited in the README.

1. Databricks workspace:



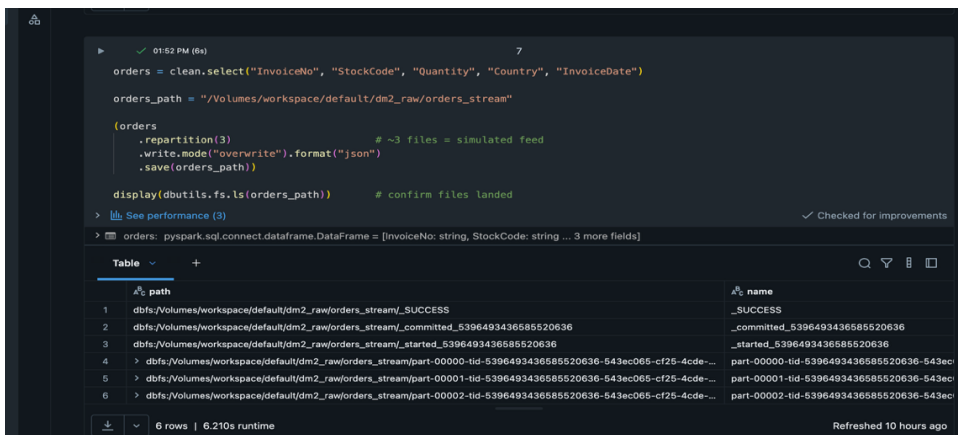
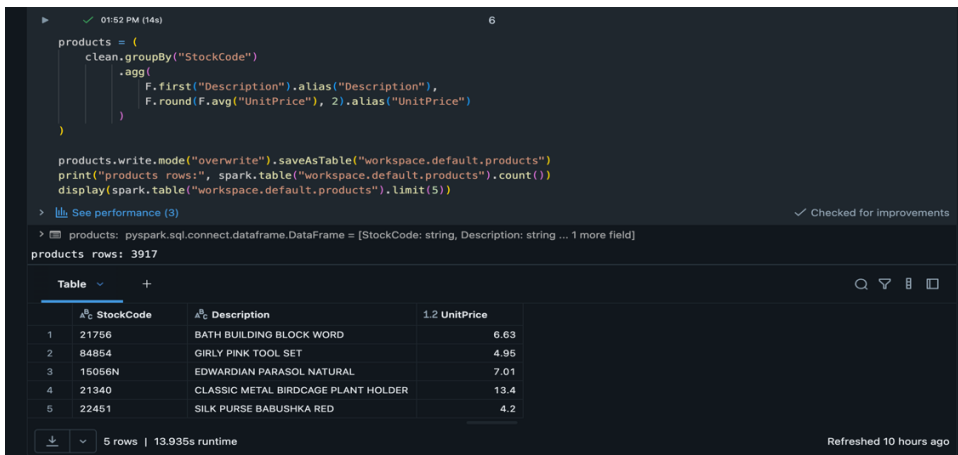
2. Catalog and Medallion tables

Catalog Explorer showing the source table (products), the raw volume (dm2_raw), and the bronze_, silver_, and gold_ tables. This evidences the two source types and the Medallion architecture.



3. The two source systems

The *products* Delta table (database source) and the *orders_stream* JSON files inside the volume (streaming source) - the two different source types being combined.



4. Bronze layer (raw ingestion)

Bronze ingestion: *bronze_products* (batch) and *bronze_orders* ingested via Auto Loader from the streaming folder.

Step 3 — Bronze layer (raw ingestion)

Land both sources as-is into Delta tables with an ingestion timestamp.

- **bronze_products** : batch read from the products table.
- **bronze_orders** : **Auto Loader** stream from the orders folder (**availableNow** trigger, so re-running ingests only new files).

```
from pyspark.sql import functions as F

(spark.table("workspace.default.products")
 .withColumn("ingest_ts", F.current_timestamp())
 .write.mode("overwrite")
 .saveAsTable("workspace.default.bronze_products"))

print("bronze_products rows:", spark.table("workspace.default.bronze_products").count())
```

> [See performance \(2\)](#) ✓ Checked for improvements

bronze_products rows: 3917

```
orders_path = "/Volumes/workspace/default/dm2_raw/orders_stream"
schema_path = "/Volumes/workspace/default/dm2_raw/_schema_orders"
chk_path = "/Volumes/workspace/default/dm2_raw/_chk_orders"

stream = (spark.readStream.format("cloudFiles")
 .option("cloudFiles.format", "json")
 .option("cloudFiles.schemaLocation", schema_path)
 .load(orders_path)
 .withColumn("ingest_ts", F.current_timestamp()))

q = (stream.writeStream
 .option("checkpointLocation", chk_path)
 .trigger(availableNow=True)
 .toTable("workspace.default.bronze_orders"))

q.awaitTermination() # waits until all current files are processed, then stops
print("done ingesting")
```

> [See performance \(1\)](#) ✓ Checked for improvements

stream: pyspark.sql.connect.dataframe.DataFrame = [Country: string, InvoiceDate: string ... 5 more fields]

done ingesting

```
print("bronze_orders rows:", spark.table("workspace.default.bronze_orders").count())
display(spark.table("workspace.default.bronze_orders").limit(10))
```

> [See performance \(2\)](#) ✓ Checked for improvements

bronze_orders rows: 507413

	Country	InvoiceDate	InvoiceNo	Quantity	StockCode	_rescued_data	ingest_ts
1	United Kingdom	01/12/10 08:26	536365	6	21730	null	2026-06-21T11:56:50.201+00:00
2	United Kingdom	01/12/10 08:26	536365	6	84029E	null	2026-06-21T11:56:50.201+00:00
3	United Kingdom	01/12/10 08:26	536365	6	85123A	null	2026-06-21T11:56:50.201+00:00
4	United Kingdom	01/12/10 08:34	536367	3	21754	null	2026-06-21T11:56:50.201+00:00
5	United Kingdom	01/12/10 08:34	536367	6	22310	null	2026-06-21T11:56:50.201+00:00
6	United Kingdom	01/12/10 08:34	536367	6	22745	null	2026-06-21T11:56:50.201+00:00
7	United Kingdom	01/12/10 08:34	536367	4	48187	null	2026-06-21T11:56:50.201+00:00
8	United Kingdom	01/12/10 08:34	536368	3	22912	null	2026-06-21T11:56:50.201+00:00
9	United Kingdom	01/12/10 08:34	536368	6	22960	null	2026-06-21T11:56:50.201+00:00
10	France	01/12/10 08:45	536370	18	21035	null	2026-06-21T11:56:50.201+00:00

5. Silver layer (clean + join)

The two sources cleaned and joined on StockCode, with revenue computed (silver_orders_enriched).

```

silver = (orders_clean.join(products_clean, on="StockCode", how="inner")
.withColumn("revenue", F.round(F.col("Quantity") * F.col("UnitPrice"), 2))
.select(
    "InvoiceNo", "StockCode", "Description",
    "Quantity", "UnitPrice", "revenue",
    F.col("Country").alias("Region"),
    "InvoiceDate"
))

silver.write.mode("overwrite").saveAsTable("workspace.default.silver_orders_enriched")

print("silver rows:", spark.table("workspace.default.silver_orders_enriched").count())
display(spark.table("workspace.default.silver_orders_enriched").limit(10))

```

silver: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 6 more fields]

silver rows: 502163

	InvoiceNo	StockCode	Description	Quantity	UnitPrice	revenue	Region	InvoiceDate
1	536373	82482	WOODEN PICTURE FRAME WHITE FINISH	6	3.11	18.66	United Kingdom	01/12/10 09:02
2	536373	844068	CREAM CUPOID HEARTS COAT HANGER	8	4.32	34.56	United Kingdom	01/12/10 09:02
3	536375	20679	EDWARDIAN PARASOL RED	6	7.09	42.54	United Kingdom	01/12/10 09:32
4	536375	82483	WOOD 2 DRAWER CABINET WHITE FINISH	2	6.95	13.9	United Kingdom	01/12/10 09:32
5	536381	22083	PAPER CHAIN KIT RETROSPOT	1	3.79	3.79	United Kingdom	01/12/10 09:41
6	536388	22922	FRIDGE MAGNETS US DINER ASSORTED	12	1.89	22.68	United Kingdom	01/12/10 09:59
7	536396	21071	VINTAGE BILLBOARD DRINK ME MUG	6	1.71	10.26	United Kingdom	01/12/10 10:51
8	536398	22835	HOT WATER BOTTLE I AM SO POORLY	8	5.9	47.2	United Kingdom	01/12/10 10:52
9	536401	21463	MIRRORED DISCO BALL	1	5.8	5.8	United Kingdom	01/12/10 11:21

6. Gold layer (business objectives)

Gold tables, one per business objective: *gold_revenue_by_region* and *gold_top_products*.

```

from pyspark.sql import functions as F

gold_region = (spark.table("workspace.default.silver_orders_enriched")
.groupBy("Region")
.agg(
    F.round(F.sum("revenue"), 2).alias("total_revenue"),
    F.countDistinct("InvoiceNo").alias("num_orders")
)
.orderBy(F.desc("total_revenue")))

gold_region.write.mode("overwrite").saveAsTable("workspace.default.gold_revenue_by_region")
display(spark.table("workspace.default.gold_revenue_by_region"))

```

gold_region: pyspark.sql.connect.dataframe.DataFrame = [Region: string, total_revenue: double ... 1 more field]

	Region	total_revenue	num_orders
1	United Kingdom	11633481.93	18019
2	Netherlands	405911.25	94
3	Germany	293069.91	457
4	France	253632.22	392
5	Spain	75863.9	90

5 rows | 6.514s runtime

```

gold_products = (spark.table("workspace.default.silver_orders_enriched")
.groupBy("StockCode", "Description")
.agg(
    F.sum("Quantity").alias("total_units"),
    F.round(F.sum("revenue"), 2).alias("total_revenue")
)
.orderBy(F.desc("total_units")))

gold_products.write.mode("overwrite").saveAsTable("workspace.default.gold_top_products")
display(spark.table("workspace.default.gold_top_products").limit(10))

```

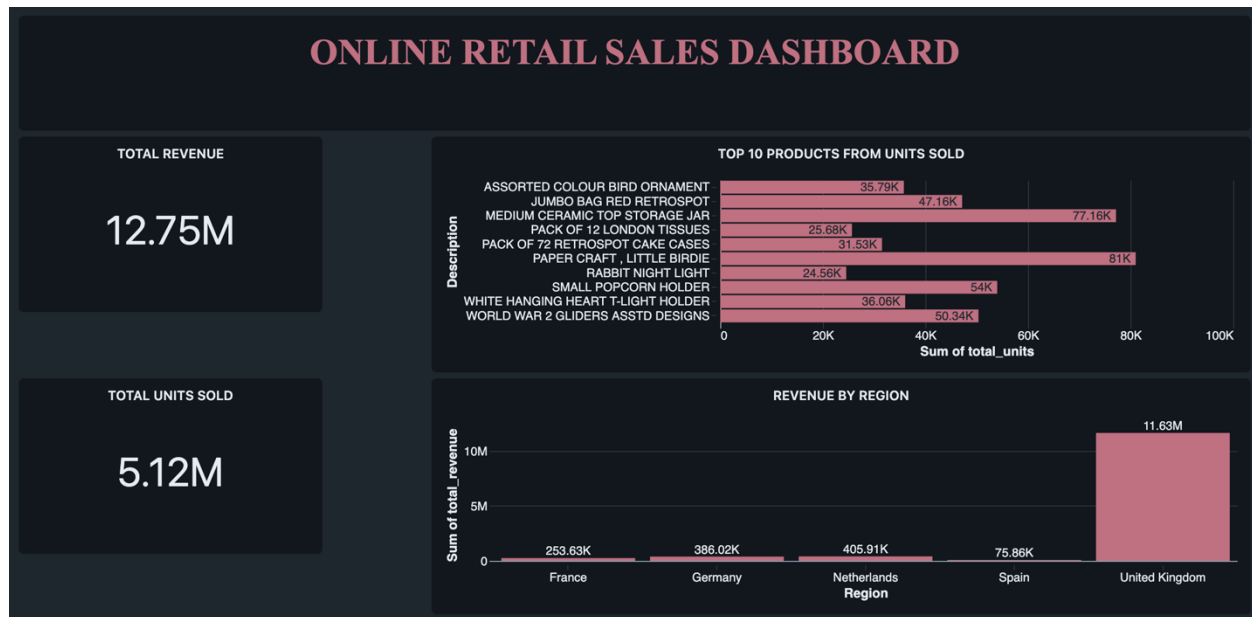
gold_products: pyspark.sql.connect.dataframe.DataFrame = [StockCode: string, Description: string ... 2 more fields]

	StockCode	Description	total_units	total_revenue
1	23843	PAPER CRAFT, LITTLE BIRDIE	80995	168469.6
2	23166	MEDIUM CERAMIC TOP STORAGE JAR	77156	114962.44
3	22197	SMALL POPCORN HOLDER	53995	56694.75
4	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	50342	16109.44
5	850998	JUMBO BAG RED RETROSPOT	47164	117910
6	85123A	WHITE HANGING HEART T-LIGHT HOLD...	36059	112864.67
7	84879	ASSORTED COLOUR BIRD ORNAMENT	35793	61563.96
8	21212	PACK OF 72 RETROSPOT CAKE CASES	31533	24280.41
9	22616	PACK OF 12 LONDON TISSUES	25675	11553.75
10	23084	RABBIT NIGHT LIGHT	24558	59184.78

10 rows | 5.175s runtime

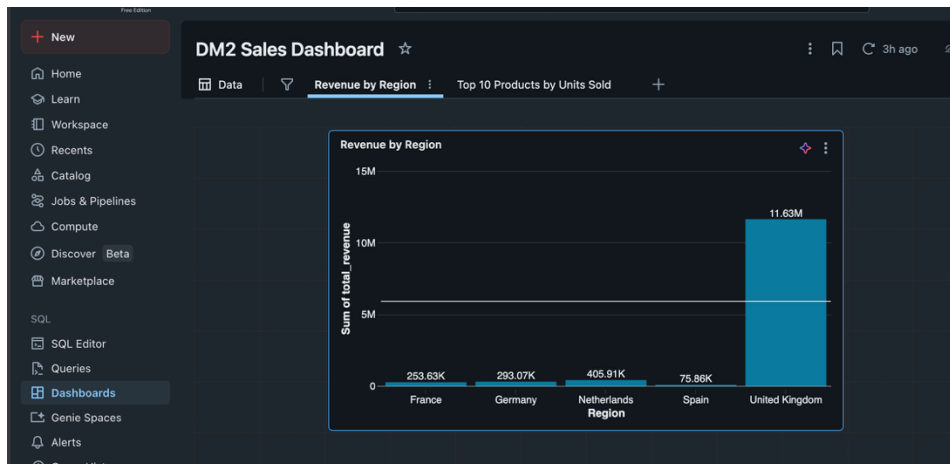
7. Dashboard

Published AI/BI dashboard “Online Retail Sales Dashboard” with the two KPI’s and Visualizations.



8. Incremental test - BEFORE

Dashboard before adding new data - Germany revenue $\approx$ 293K.



9. Incremental test - AFTER

Dashboard after appending new orders and re-running the pipeline — Germany revenue $\approx$ 386K. This demonstrates that adding data to a source propagates through every layer to the dashboard, as required.

